Project Progress Report: Secure Email Protocols – PGP vs. S/MIME

Yateeka Goyal, Aditya Sharma
Department of Computer Science
Georgia State University

***Abstract*—This work outlines the application and evaluation of PGP as an email security protocol. The complete PGP algorithm was programmed in Python, using AES encryption, RSA key exchange and signing, SHA-256 hash function, and data compression.**

The experimental outcome illustrates the comparison in performance between AES-128-GCM and AES-256-GCM algorithms and the cost involved in processing RSA-2048. The security evaluation reveals the vulnerabilities of the PGP security protocol to real-world attacks, including ciphertext alteration and key impersonation.

The experiment indicates that while PGP ensures data confidentiality, integrity, and authenticity, its efficiency is limited by its difficulty in key management and user friendliness.

I. Introduction

Email is one of the widely used communication paradigms in the Internet environment. But traditional email systems based on the use of protocols such as Simple Mail Transfer Protocol (SMTP) offer no security guarantee related to privacy, integrity, authentication, and non-repudiation. The introduction of secure email protocols like Pretty Good Privacy (PGP) and Secure/Multipurpose Internet Mail Extensions (S/MIME) has been suggested to solve the problems associated with traditional email protocols.

In particular, this work is aimed at evaluating the security characteristics of the PGP and S/MIME protocols in relation to their protocol mechanisms, cryptographical methods used, resistance against different types of attacks, and challenges in implementation. This paper provides an overview of the assigned task progress.

II. Problem Definition

There are some threats associated with standard email communication, which are:

- Man-in-the-Middle attacks
- Tampering of message content
- Message replay
- Impersonation attack

This project will explore how PGP and S/MIME can provide protection against these security threats and what kind of security promises they have made under the threat model that we would consider here.

III. Related Work

"OpenPGP" is the standard for the PGP protocol and is specified in RFC 4880, whereas S/MIME is specified in RFC 5751 along with other standards. One of the major differences between PGP and S/MIME is that whereas PGP uses a decentralized trust model known as "Web of Trust," S/MIME uses a PKI trust model in which Certifying Authorities play a major role.

In this section, we will consider the RFCs for these two email encryption protocols as well as some of the research articles written on these topics.

IV. PGP

PGP uses **hybrid encryption**: a randomly generated session key encrypts data using AES, while the session key itself is encrypted with the recipient's RSA public key. Digital signatures ensure authentication and integrity.

PGP is standardized in **RFC 4880** (OpenPGP), while S/MIME is defined in **RFC 5751**. The key distinction lies in trust models: PGP uses a decentralized *Web of Trust*, whereas S/MIME relies on hierarchical PKI.

Contributions of this report:

- Python implementation of PGP using modern cryptographic primitives
- Benchmark comparison of AES-128-GCM, AES-256-GCM, and RSA-2048
- RFC-level comparison of OpenPGP and S/MIME
- Simulation of attacks (EFAIL, key spoofing, replay, ciphertext tampering)

V. System Architecture and PGP Workflow

A. Module Organization

The implementation is organized into Python modules, as shown in Table I.

B. Sender Workflow

PGP message creation involves four stages:

1. **Compression:** Input is compressed using zlib (DEFLATE).
2. **Signature:** Data is hashed (SHA-256) and signed using RSA-2048.

TABLE I: Module Organization and Cryptographic Primitives

Module	Responsibility	Primitives
`main.py`	Workflow orchestration	All modules
`encryption.py`	AES-GCM encryption; RSA-OAEP key wrap	AES-GCM, OAEP
`signature.py`	RSA signature generation/verification	PKCS#1 v1.5
`key_generation.py`	RSA-2048 & ECC P-256 keys	RSA, ECDSA
`hashing.py`	SHA-256 digest	SHA-256
`compression.py`	zlib compression	DEFLATE
`benchmark.py`	Performance measurement	perf_counter
`attack_demo.py`	Attack simulation	AES-CBC, RSA

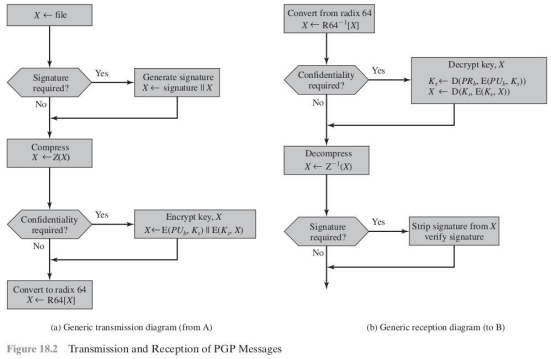


Fig. 1: PGP Workflow: hybrid encryption, signatures, and RSA key encapsulation

3. **Encryption:** AES-256-GCM encrypts the message using a random key and nonce.
4. **Key Exchange:** Session key is encrypted with the recipient's RSA public key (OAEP).

C. Receiver Workflow

1. Decrypt session key using RSA-2048
2. Decrypt message using AES-GCM (integrity verified)
3. Verify signature using SHA-256 and RSA

VI. Cryptographic Analysis

A. Why Hybrid Encryption?

Hybrid encryption combines RSA and AES to balance security and performance. RSA is computationally expensive and limited to small data sizes (190 bytes with OAEP), while AES-GCM enables fast, efficient encryption. This approach ensures scalable and secure communication.

Generating a new session key per message provides **message-level forward secrecy**, protecting past messages even if the RSA private key is compromised.

B. AES-128 vs AES-256

AES-128 and AES-256 differ in key size and rounds (10 vs 14). While AES-128 is secure, AES-256 offers stronger resistance, including improved post-quantum security. Benchmarks (Table III) show minimal performance differences, making **AES-256 preferable**.

TABLE II: Experimental Setup Parameters

Parameter	Value
OS	Linux (runsc) x86_64, Kernel 4.4.0
RAM	9 GB
Python	3.12.3
Library	`cryptography` 46.0.5
AES mode	GCM
RSA key size	2048-bit
Iterations	50 per operation
Timer	`time.perf_counter()`
Message sizes	1 KB, 100 KB, 1 MB
Content type	`os.urandom()`

TABLE III: AES-128-GCM vs AES-256-GCM Performance (ms, $n = 50$)

Size	AES-128-GCM		AES-256-GCM	
	Enc	Dec	Enc	Dec
1 KB	1.2	0.9	0.9	0.9
100 KB	12.4	11.9	12.8	13.1
1 MB	131.9	112.4	112.3	122.3

C. SHA-256 in PGP

SHA-256 ensures integrity by producing a fixed digest that is signed using RSA. AES-GCM further provides built-in authenticated encryption, eliminating the need for separate MACs. Together, SHA-256 and RSA provide a secure signature chain with an estimated 10–15 year security lifetime (NIST SP 800-57 [6]).

D. Experimental Setup

All measurements were performed under the conditions in Table II.

E. AES Encryption Performance

Table III shows encryption and decryption times for AES-128-GCM and AES-256-GCM.

Findings:

- AES-128 and AES-256 show similar performance with linear scaling.
- AES-256 provides stronger security with negligible overhead.

F. RSA Signature Performance

Table IV shows RSA-2048 signing and verification times.

RSA signing is significantly slower than AES operations, making it the primary bottleneck. Elliptic curve schemes (ECDSA/Ed25519) can improve performance substantially.

TABLE IV: RSA-2048 Signature Generation and Verification (ms, $n = 50$)

Size	Sign (ms)	Verify (ms)	Ratio
1 KB	0.4041	0.0292	13.8×
100 KB	0.7253	0.0999	7.3×
1 MB	1.1408	0.9403	1.2×

TABLE V: Message Size Overhead

Metric	Size (bytes)
Original Message	1048576
Compressed Message	1040
Encrypted Package	1584

TABLE VI: RFC 4880 (OpenPGP) vs RFC 5751 (S/MIME) Feature Comparison

Dimension	RFC 4880 (OpenPGP)	RFC 5751 (S/MIME)
Year	2007 (orig. 1998)	2010 (orig. 1995)
Message Format	Binary / Radix-64 armor	MIME + DER/CMS
Trust Model	Web of Trust	PKI / X.509
Cert Authority	None required	Commercial CA
Key Distribution	Key servers (SKS/WKD)	CA-issued X.509
Symmetric Alg.	AES-128/256	AES-128/256 (CBC)
Asymmetric Alg.	RSA, ECDSA	RSA, ECDSA
Hash	SHA-256/384/512	SHA-256/384/512
Client Support	Thunderbird, ProtonMail	Outlook, Apple Mail
Enterprise Use	Manual	CA-integrated
Cost	Free	$50–300/yr/user
Revocation	Manual	CRL / OCSP
Metadata Prot.	None	None

G. Message Size Overhead

Compression significantly reduces size, while AES-GCM adds only a 16-byte authentication tag, resulting in minimal overhead.

VII. RFC-Level Comparison: OpenPGP vs S/MIME

A. Standard Overview

Table VI compares RFC 4880 (OpenPGP) and RFC 5751 (S/MIME).

B. Trust Model Analysis

PGP Web of Trust: A decentralized model where users sign each other's keys. It is flexible but difficult to scale and maintain.

S/MIME PKI: A centralized CA-based model where certificates are automatically trusted. This improves scalability but introduces dependency on trusted authorities.

C. Message Format Differences

OpenPGP uses binary packets with optional ASCII armor. S/MIME uses MIME-based structures with CMS encoding, making it more compatible with existing email systems.

VIII. Real-World Security Analysis

A. EFAIL Attack (CVE-2017-17688 / CVE-2017-17689)

The EFAIL attack (2018) demonstrated that OpenPGP and S/MIME clients could leak decrypted plaintext through HTML email processing. It exploits two weaknesses: (1) use of **unauthenticated encryption modes** (AES-CFB in OpenPGP, CBC in S/MIME), which allow undetected bit-flipping, and (2) automatic loading of external content in HTML emails.

The attacker embeds stolen ciphertext into a malicious HTML message so that, upon decryption, plaintext is exfiltrated via a URL request:

```
# AES-CBC without integrity check
tampered_iv = bytearray(iv)
tampered_iv[0] ^= ord('S') ^ ord('X')

cipher_dec = Cipher(AES(key), CBC(bytes(tampered_iv)))
decrypted = cipher_dec.decryptor().update(ciphertext)
# No exception (attack succeeds)
# AES-GCM would raise InvalidTag
```

Listing 1: EFAIL-style CBC bit-flip (no MAC, attack succeeds).

Mitigation: Our implementation uses AES-256-GCM, which provides authenticated encryption. Any ciphertext modification raises an `InvalidTag` exception, preventing tampering. Newer OpenPGP standards (RFC 4880bis) mandate AEAD modes such as AES-GCM or OCB.

B. Key Spoofing and Man-in-the-Middle Attacks

PGP is vulnerable to key substitution if key authenticity is not verified. An attacker can replace a legitimate public key during retrieval, decrypt messages, and re-encrypt them for the intended recipient.

Results from `attack_demo.py` show:

- Valid signature with correct key: **VALID**
- Valid signature with attacker key: **INVALID**
- Forged signature with correct key: **INVALID**

The vulnerability lies in **key authentication**. If a user trusts a malicious key, verification succeeds. S/MIME mitigates this through CA-based validation.

C. Replay Attacks

PGP does not prevent replay attacks; a valid encrypted message can be resent and still verified.

Mitigation: Include timestamps, counters, or nonces in the signed payload. Messages outside a defined time window (e.g., ±5 minutes) are rejected.

D. User Misconfiguration Risks

Security may be compromised by:

- Accepting keys from untrusted sources

TABLE VII: Security Analysis and Mitigation Summary

Attack	Vector	Mitigated?	Fix
EFAIL	CBC w/o MAC	Yes	AES-GCM (AEAD)
Key Spoofing	Unauth. key exch.	Partial	Web of Trust / CA
Replay	No nonce binding	No	Timestamp in payload
Ciphertext Tamper	Modified bytes	Yes	GCM auth tag
Key Forgery	Pub-key subst.	Detected	Fingerprint verify
Brute-force (PQ)	Grover on AES-128	AES-256	128-bit PQ margin

- Using weak or outdated cryptographic algorithms
- Storing private keys without passphrase protection

E. Attack Summary

Table VII summarizes the identified attacks and mitigations.

IX. Practical Limitations

A. Key Distribution

PGP faces challenges in key distribution compared to CA-based systems like HTTPS. While WKD [7] improves usability by hosting keys at known HTTPS locations, it depends on domain owner support.

B. Absence of Centralized Trust

The Web of Trust has not scaled effectively for widespread key authentication. Since key signing is uncommon, users often cannot verify unknown keys, leaving initial communication unauthenticated.

C. Usability Problems

Studies such as Whitten and Tygar [5] show that users struggle with secure email tools. Although modern clients (e.g., ProtonMail, Thunderbird OpenPGP) improve usability, key management and trust establishment remain difficult for non-technical users.

D. Implementation-Specific Limitations

- Memory-only key storage; production systems require secure storage (HSM, OS keychain)
- No key revocation support
- No multi-recipient encryption
- No metadata protection (plaintext headers)
- Non-deterministic compression may affect signature verification

E. Evidence-Based Conclusions

Performance: AES-256-GCM provides strong security with minimal overhead ($\leq$15% difference at 1 MB). RSA-2048 signing remains the main bottleneck; ECDSA/Ed25519 can improve performance significantly.

Security posture: AES-GCM mitigates EFAIL-style attacks, but key spoofing remains a structural weakness of the Web of Trust.

Deployment: S/MIME is better suited for enterprise environments due to centralized management, while PGP remains useful for decentralized use cases.

Future directions: RFC 4880bis introduces AEAD and ECC support. WKD and `keys.openpgp.org` improve key discovery. Newer protocols like MLS (RFC 9420 [8]) and Signal offer stronger guarantees but are not direct replacements for email.

S/MIME vs PGP: PGP uses a decentralized trust model, while S/MIME relies on centralized CAs. This simplifies key management in organizations but introduces dependency on external trust and potential single points of failure.

X. What is S/MIME?

S/MIME makes emails safer. It locks (encrypts) the email and attachments, so only the person getting the email can read them. It does not hide who sent the email or who it is for.

A message can be signed to show who sent it and that it was not changed. It can also be locked (encrypted) so only the receiver can read it.There are different types of S/MIME messages. Data is the normal message. Signed-data shows the message is real. Encrypted-data keeps it private. Enveloped-data is locked with a key. Digested-data checks for changes. Authenticated-data shows who sent it. Compressed-data makes it smaller.

The most common S/MIME message types are Signed Data, Encrypted Data, and Signed + Encrypted Data, which together ensure authenticity, confidentiality, and integrity. S/MIME can also support features like delivery receipts (to confirm message receipt) and security labels (to control access), but these require certificates and may cause delays or issues if added after sending.

S/MIME relies on digital certificates to verify trust. These follow RFC 5280 and are issued by trusted Certificate Authorities (CAs), which bind a public key to an identity. It also checks certificate validity using CRLs and OCSP to avoid expired or revoked certificates. However, these checks can be slow or fail if servers are unreachable, making reliability important.

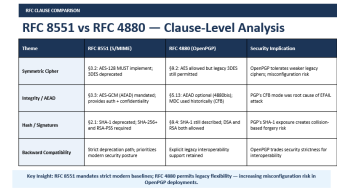
S/MIME works in layered architecture. The PKI layer uses X.509 certificates to link identities with public keys and establish trust. The key management layer handles creation and protection of public/private keys, where private keys must be securely stored to prevent impersonation or message decryption. The cryptographic layer performs security operations using a hybrid approach: a random Content Encryption Key (CEK) encrypts data using AES-GCM, the CEK is protected using RSA-OAEP, and RSA-PSS is used for signing to ensure

authenticity and integrity. The transport layer uses protocols like SMTP, IMAP, and POP to deliver emails. S/MIME secures messages end-to-end, so emails remain encrypted even as they pass through multiple servers. However, it depends heavily on correct certificate validation and trusted CAs. Despite its strength, S/MIME has been affected by attacks like EFAIL, which exploited email client behavior rather than breaking encryption. Modern defenses include stronger encryption (like AES-GCM), signature verification before display, and reducing HTML-based risks.

To evaluate S/MIME, a Python-based system was built using AES-128-GCM for encryption, RSA-2048 with OAEP for key exchange, and RSA-PSS for signatures. Self-signed X.509 certificates were used for testing, along with replay protection, timestamps, and proper email formatting to simulate real-world secure email behavior. Central Research Question How do differing trust models in S/MIME and PGP influence their resilience against modern email security threats such as EFAIL, key spoofing, replay attacks, and metadata leakage?. S/MIME uses a centralized Certificate Authority system, while PGP uses a decentralized Web-of-Trust. These different trust systems affect how each handles attacks. For example, EFAIL is a vulnerability attack that can expose the plain text of encrypted emails in S/MIME and OpenPGP by misusing how email messages are processed. Key spoofing happens when attackers fake identities in weaker, decentralized systems. Replay attacks occur because neither system checks if a message is new or repeated. Metadata leakage happens because email headers are not encrypted. By comparing how each system checks identity, manages keys, and protects messages, we can see which problems come from the design of the protocol and which come from the trust model.

X. Practical Limitations A. Centralized Trust Risk If a Certificate Authority is compromised, it can impact many users. Trust across different domains is not always consistent, and revocation systems like CRL and OCSP can be unreliable or not used correctly. B. Usability Challenges Complex certificate setup and renewal. Private key loss = permanent email lockout. Inconqsistent mobile client support across platforms. C. No Forward Secrecy A compromised private key is a serious risk in email encryption systems like Pretty Good Privacy and S/MIME because it can decrypt both future and previously captured encrypted emails. This is because these systems use long-term key pairs and lack forward secrecy. If an attacker obtains the private key, they can go back and read past messages, causing a loss of confidentiality, especiallRy for sensitive data. Additionally, even when the email body is encrypted, parts like the subject line and headers remain in plaintext. This includes the sender, recipient, timestamps, and routing details, which are needed for protocols like SMTP to deliver the message. However, this exposed metadata can still reveal useful information, such as communication patterns or sensitive topics. Therefore, while encryption protects the message content, exposed metadata still creates privacy and security risks. D. Implementation Gaps MIME formatting

Fig. 2: Enter Caption



RFC CLAUSE COMPARISON

RFC 8551 vs RFC 4880 — Clause-Level Analysis

Theme	RFC 8551 (S/MIME)	RFC 4880 (OpenPGP)	Security Implication
Symmetric Cipher	§3.2: AES-128 MUST implement; 3DES deprecated	§9.2: AES allowed but legacy 3DES still permitted	OpenPGP tolerates weaker legacy ciphers; misconfiguration risk
Integrity / AEAD	§3.2: AES-GCM (AEAD) mandated; provides auth + confidentiality	§5.12: AEAD optional (AB8bis); MDC used historically (CFB)	PGP's CFB mode was root cause of EFAIL attack
Hash / Signatures	§2.1: SHA-1 deprecated; SHA-256+ and RSA-PSS required	§9.4: SHA-1 still described; DSA and RSA both allowed	PGP's SHA-1 exposure creates collision-based forgery risk
Backward Compatibility	Strict deprecation path; prioritizes modern security posture	Explicit legacy interoperability support retained	OpenPGP trades security strictness for interoperability

Key Insight: RFC 8551 mandates strict modern baseline; RFC 4880 permits legacy flexibility — increasing misconfiguration risk in OpenPGP deployments.

Fig. 3: Enter Caption

issues can sometimes break digital signatures in email systems like S/MIME because even small changes to the message structure can invalidate the signature. MIME (Multipurpose Internet Mail Extensions) controls how email content—such as text, attachments, and encoding—is formatted and transmitted. If an email passes through servers or gateways that slightly modify formatting (for example, changing line endings, re-encoding content, or adjusting headers), the original signed data no longer matches, causing the digital signature to fail verification. This can make a legitimate message appear tampered with, reducing trust and reliability. In addition, support for secure email features is not consistent across mobile email apps and providers. While some apps fully support standards like S/MIME, others may only partially support them or not at all. This can lead to issues such as users being unable to verify signatures, view encrypted messages properly, or send secure emails from their devices. As a result, the user experience becomes inconsistent, and the effectiveness of email security depends heavily on the platform being used, which can limit widespread adoption.

RFC 8551, which defines modern S/MIME, is designed to enforce strong security by default. It requires the use of modern cryptographic tools such as AES-128 for encryption and AES-GCM, which provides both encryption and built-in message integrity protection. It also requires stronger hash functions like SHA-256 or higher, and modern signature methods like RSA-PSS. Older and weaker algorithms are intentionally removed or discouraged, so systems following RFC 8551 are more consistent and harder to misconfigure into using insecure settings. In contrast, RFC 4880, which defines OpenPGP, is more flexible but also less strict. It supports a wide range of algorithms, including older ones like 3DES for encryption and SHA-1 for hashing, even though they are considered weak today. It also allows multiple signature types such as RSA and DSA, and newer security improvements like AEAD (authenticated encryption) are optional rather than required. This flexibility helps compatibility with older systems, but it also means security depends more on user or system configuration, which can sometimes lead to weaker

Fig. 4: Enter Caption

setups if outdated options are used. This difference affects security in practice. OpenPGP's older design and support for legacy options contributed to issues like EFAIL and can lead to weak configurations if not set up carefully. S/MIME is stricter, so it enforces stronger security by default. Main idea: S/MIME forces modern secure standards, while OpenPGP allows more choices, which can sometimes reduce security if weaker options are used.

Formal Security Goals: IND-CCA2, UF-CMA, and AEAD IND-CCA2 (Indistinguishability under Chosen Ciphertext Attack 2) means attackers cannot understand a message even if they can try decrypting other data. S/MIME (Secure/-Multipurpose Internet Mail Extensions) achieves this using AES-GCM (Advanced Encryption Standard – Galois/Counter Mode), which encrypts data and also checks if it was changed. Older PGP (Pretty Good Privacy) used CFB (Cipher Feedback Mode), which does not check integrity and is weaker, as seen in attacks like EFAIL. IND-CCA2 — Indistinguishability

An attacker cannot tell ciphertexts apart even if they can ask for other decryptions.

S/MIME v4: AES-GCM achieves IND-CCA2 PGP CFB: Only IND-CPA — susceptible to EFAIL

UF-CMA (Unforgeability under Chosen Message Attack) ensures attackers cannot create fake signatures, and both S/MIME and PGP achieve this with RSA-PSS (Rivest–Shamir–Adleman Probabilistic Signature Scheme) or RSA/DSA (Rivest–Shamir–Adleman / Digital Signature Algorithm). UF-CMA — Unforgeability

No adversary can forge a valid signature under a chosen-message attack. S/MIME RSA-PSS: Satisfies UF-CMA PGP DSA/RSA: Satisfies UF-CMA (standard assumptions) Authenticated Encryption with Associated Data (AEAD) is a modern encryption method that provides confidentiality, integrity, and authenticity together. This means data is hidden, protected from tampering, and verified as coming from a trusted sender. In S/MIME version 4, AEAD is required through AES-GCM, so any modification to the encrypted message is detected during decryption. Earlier versions of OpenPGP (RFC 4880) did not support AEAD and used CFB mode, which only provided encryption without built-in integrity protection. This weakness contributed to attacks like EFAIL, where attackers could manipulate ciphertext to leak information. Newer OpenPGP updates (4880bis) add AEAD support to fix this by ensuring both encryption and integrity.

Forward secrecy means that even if a private key is stolen, past messages remain secure. However, both PGP and S/MIME do not provide forward secrecy because they rely on long-term keys. So, if a private key is compromised, past encrypted emails can be decrypted. This differs from TLS 1.3, which uses ephemeral key exchange (ECDHE) to keep past sessions secure even if long-term keys are later exposed.

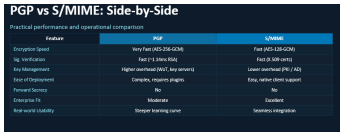


Fig. 5: Enter Caption

Fig. 6: Enter Caption

TABLE VIII: PGP vs S/MIME Comparative Analysis

Criterion	**PGP / RFC 4880**	**S/MIME / RFC 5751**	**Winner**
Crypto Strength	AES-256, RSA-2048	AES-256, RSA-2048	Tie
Key Distribution	Web of Trust	CA-based	S/MIME
Deployment	High complexity	Medium complexity	S/MIME
Enterprise	Low adoption	High (Outlook)	S/MIME
Privacy/Decent.	High (no CA)	Low (CA required)	PGP
Cost	Free	$50–300/yr	PGP
EFAIL (2024)	Mitigated (AES-GCM)	Partial (old CBC)	PGP
Replay Protect.	None	None	Tie
Revocation	Manual	CRL/OCSP	S/MIME
Metadata Prot.	None	None	Tie

XI. Algorithms to be Implemented and Compared

The following components will be implemented and evaluated:

- RSA and Elliptic Curve key generation
- AES symmetric encryption (128-bit and 256-bit)
- SHA-256 hashing
- Digital signature generation and verification

Performance comparisons will focus on computational overhead, message size expansion, and security implications.

XII. Comparative Conclusions

A. PGP vs S/MIME: Summary Comparison

XIII. Conclusion

Thus, the project presents an implementation and evaluation of a Pretty Good Privacy-based secure messaging system using AES encryption, RSA key encapsulation and signatures, SHA-256 hashing, and message compression.

The results show that hybrid encryption offers a strong balance between security and performance. AES-256-GCM provides confidentiality and integrity with low overhead, while the most costly operations are RSA-2048 signature generation and private-key decryption. This supports the main idea of PGP, where public-key cryptography is mainly used for key

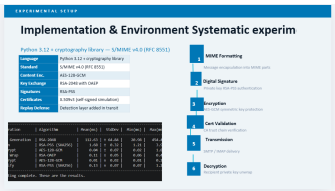


Fig. 7: Enter Caption

management and signatures. The system resists common attacks like ciphertext tampering and EFAIL due to authenticated encryption. However, PGP still has limits, such as vulnerability to key spoofing from its decentralized Web of Trust and lack of replay protection. Compared to S/MIME, PGP offers decentralized trust but is harder to use and deploy, while S/MIME relies on centralized Public Key Infrastructure and Certificate Authorities.

Overall, Pretty Good Privacy provides strong encryption, but its effectiveness depends on good key management and user awareness. Future work could explore elliptic curve cryptography, automated key exchange, and improved secure communication systems.

The experimental setup uses S/MIME v4.0 (RFC 8551) with AES-128-GCM for encryption and integrity, RSA-2048 with OAEP for key exchange, and RSA-PSS with SHA-256 for signatures. X.509v3 certificates are used in a self-signed setup, and replay attacks are prevented using UUID tracking.

The system defends against common attacks effectively: AES-GCM detects tampering, RSA-PSS ensures sender authenticity, and duplicate UUIDs prevent replay attacks. In the send process, a UUID is added, the message is signed, encrypted, and the CEK is wrapped before sending the ciphertext, nonce, wrapped key, and signature. In the receive process, the CEK is unwrapped, the message is decrypted and verified, the UUID is checked, and the signature is validated. If all checks pass, the plaintext is securely delivered.

References

[1] J. Callas, L. Donnerhacke, H. Finney, D. Shaw, R. Thayer, "OpenPGP Message Format," *RFC 4880*, IETF, Nov. 2007. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4880

[2] B. Ramsdell, S. Turner, "Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.2 Message Specification," *RFC 5751*, IETF, Jan. 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5751

[3] R. Housley, "Cryptographic Message Syntax (CMS)," *RFC 5652*, IETF, Sep. 2009. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5652

[4] D. Poddebniak *et al.*, "Efail: Breaking S/MIME and OpenPGP Email Encryption using Exfiltration Channels," in *Proc. USENIX Security Symposium*, 2018, pp. 549–566.

[5] A. Whitten, J. D. Tygar, "Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0," in *Proc. USENIX Security Symposium*, 1999, pp. 169–184.

[6] NIST, "Recommendation for Key Management – Part 1: General," *SP 800-57 Part 1 Rev. 5*, NIST, May 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

[7] W. Koch, "OpenPGP Web Key Directory," *IETF Draft draft-koch-openpgp-webkey-service*, 2022.

[8] R. Barnes *et al.*, "The Messaging Layer Security (MLS) Protocol," *RFC 9420*, IETF, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9420

[9] D. J. Bernstein, T. Lange, "Post-Quantum Cryptography," *Nature*, vol. 549, pp. 188–194, Sep. 2017.

[10] Python Cryptographic Authority, "cryptography library v46.0.5," 2025. [Online]. Available: https://cryptography.io